Compiled vs Interpreted Languages

Compiled languages

In compiled languages, the **entire source code** is translated into **machine code before the program runs**.
This translation is done by a **compiler**.

How it works:

1. You write source code
2. The compiler converts it to machine code
3. The operating system runs the executable

Key points:

- Faster execution because code is already machine code
- Errors are detected at compile time
- Platform-dependent executables

Examples: C, C++

Interpreted languages

In interpreted languages, the code is **executed line by line at runtime** by an **interpreter**.

How it works:

1. You write source code
2. The interpreter reads and executes each line while running

Key points:

- Slower because code is translated during execution
- Errors appear at runtime
- More flexible and easier to test/debug
- Usually platform-independent

Examples: Python, JavaScript

Where does C# fit?

C# uses a **two-step compilation model**, so it is considered **hybrid**.

Step 1: Compilation to IL

- C# code is compiled by the C# compiler into **IL (Intermediate Language)**
- IL is **not machine code**
- This makes C# programs **platform-independent**

Step 2: JIT Compilation at Runtime

- When the program runs, the **CLR (Common Language Runtime)** converts IL into **machine code**
- This is done using **JIT (Just-In-Time) compilation**
- Only the needed parts are compiled, improving performance

Why C# uses this approach

- Combines **performance** of compiled languages
- Keeps **portability** and **safety**
- Enables runtime features like:
  - Garbage collection
  - Type safety
  - Exception handling

1. Implicit Casting

What it is:
Automatic conversion done by the compiler.

When it works:
From **smaller** → **larger** data types (safe conversion).

Characteristics:

- No casting operator needed
- No data loss
- Compile-time safe
- Cannot throw exceptions

Example:

```
int x = 10; double y = x;
```

---

2. Explicit Casting

What it is:
Manual conversion using a casting operator.

When it's required:
From **larger** → **smaller** data types.

Characteristics:

- Uses `(type)` syntax
- May cause data loss
- No exception by default (unless `checked`)
- Programmer takes responsibility

Example:

```
double d = 9.8; int x = (int)d; // result: 9
```

---

3. Convert

What it is:
Conversion using the `Convert` class.

When it's used:
Safe conversion with **runtime checking**.

Characteristics:

- Performs range checking
- Throws exceptions on overflow or invalid data
- Can handle `null`
- More controlled than casting

Example:

```
double d = 9.8; int x = Convert.ToInt32(d); // rounds to 10
```

4. Parse

What it is:
Used to convert **string** → **value type**.

When it's used:
When input is text (e.g., user input).

Characteristics:

- Works only with strings
- Throws exception if format is invalid
- No safety unless wrapped in try-catch

Example:

```
string s = "123"; int x = int.Parse(s);
```

self study

1️⃣ Physical reality: what memory *really* is

At the **hardware level**, RAM is just:

- A **linear array of bytes**
- Each byte has an **address**
- No concept of stack, heap, object, class, variable

```
Address → Data 0x1000 → 0x3A 0x1001 → 0xFF ...
```

📌 **Stack and heap do NOT exist in hardware**
They are **abstractions built by software**.

2️⃣ How memory is divided (REALISTIC view)

🔹 Physical memory

- Actual RAM chips

🔹 Virtual memory (OS illusion)

Each process gets:

```
0x00000000 ┌──────────────────────┐
           │ Code (text) segment  │
           │ Static / data segment│
           │ Heap  (↑ grows upward)
           │                      │
           │                      │
           │ Stack (↓ grows downward)
           └──────────────────────┘
```

📌 This layout is **logical**, not physical.

3️⃣ Is stack a “stack data structure”?

✔️ Conceptually: YES

❌ Implementation-wise: NO linked list, NO nodes

What the stack *really* is:

- A **contiguous memory region**
- Managed by **stack pointer (SP)** register
- Push = `SP -= size`
- Pop = `SP += size`

📌 This is why stack is:

- Extremely fast
- Deterministic
- No fragmentation

So:

> Stack behaves like the **stack ADT**, but is implemented as **pointer arithmetic**

4️⃣ Is heap a “heap data structure”?

❌ Absolutely NOT

Heap memory:

- Is **not ordered**
- Is **not a tree**
- Is just a **free memory pool**

The name is historical.

Heap data structure:

- Binary tree
- Priority-based
- Algorithmic concept

📌 Zero relation besides the name.

5️⃣ Heap internals (OS level)

Heap is:

- A large memory region
- Managed by:
  - `malloc/free` (C)
  - CLR GC (C#)
  - `new/delete` (C++)

Heap allocator responsibilities:

- Find free block
- Handle fragmentation
- Track allocated blocks

Basic strategies:

- Free lists
- Bump pointer
- Segregated lists

6️⃣ Process stack vs CLR managed heap

Each **thread** has:

- Its **own stack**
- Stack created by OS

Each **process** has:

- One **managed heap** (CLR)

```
Thread 1 → Stack 1 Thread 2 → Stack 2 Process → One CLR Heap
```

7️⃣ What exactly is stored on the stack?

Stack stores **execution state**, not objects.

Stack frame contains:

- Return address
- Saved registers
- Parameters
- Local variables
- Spill slots

Example:

```
void Foo(int a) { int x = 5; }
```

Stack frame:

```
| return addr | | old rbp | | a | | x |
```

📌 Stack does NOT know about classes or objects.

8️⃣ CLR heap internals (REAL STUFF)

CLR heap is divided into:

🟦 Small Object Heap (SOH)

- Objects < ~85 KB
- Allocated via **bump pointer**

```
heap_top → [object][object][object]
```

🟧 Large Object Heap (LOH)

- Objects ≥ 85 KB
- No compaction (mostly)

🟩 Generations

Generation	Purpose
Gen 0	Short-lived
Gen 1	Medium
Gen 2	Long-lived

📌 GC relies on **survival rate heuristics**

9️⃣ How CLR allocates an object (step-by-step)

```
Person p = new Person();
```

What happens:

1. CLR knows object size at JIT time
2. Checks Gen0 free space
3. Allocates:
   - Object header
   - Method table pointer
   - Fields
4. Moves heap pointer
5. Returns address (reference)

```
Heap: [ObjHeader][MT ptr][field1][field2]
```

Stack:

```
p → heap address
```

📌 Allocation ≈ 3 CPU instructions

🔟 Why references go on stack but objects on heap

Stack rule:

> Stack cannot contain data that may outlive the function

Heap rule:

> Heap objects live independently of call stack

So:

```
return new Person();
```

Impossible to store on stack → function returns → stack destroyed

What does “C# is managed code” actually mean?

> **Managed code = code whose execution, memory, and lifetime are controlled by a runtime (CLR), not directly by the OS or hardware.**

1️⃣ Unmanaged code vs Managed code (at the lowest level)

🔴 Unmanaged code (C / C++)

- Compiled **directly to machine code**
- OS loads it and runs it
- Programmer is responsible for:
  - Memory allocation/free
  - Object lifetime
  - Pointer safety
  - Resource cleanup

Example:

```
int* p = malloc(100); free(p); // if you forget → memory leak
```

No one is watching you.

🟢 Managed code (C#)

C# is **not executed directly** by the CPU.

Pipeline:

```
C# source ↓ IL (Intermediate Language) ↓ CLR (JIT) ↓ Machine code
```

CLR sits **between your code and the OS**.

2️⃣ What exactly does the CLR “manage”?

🔹 1. Memory management (BIGGEST one)

You **never call malloc/free**.

CLR:

- Allocates objects
- Tracks references
- Detects unreachable objects
- Frees memory automatically (GC)

This is what people usually mean by “managed”.

```
var obj = new Person(); // No delete. Ever.
```

🔹 2. Object lifetime & safety

CLR guarantees:

- No dangling pointers
- No use-after-free
- No double-free
- Type safety

Why?

- References ≠ raw addresses
- CLR controls access

🔹 3. Type safety & verification

Before execution:

- CLR **verifies IL**
- Ensures:
  - Stack is balanced
  - Types match
  - No invalid jumps
  - No illegal memory access

Unmanaged code:

- CPU will happily execute garbage

Managed code:

- CLR refuses to run invalid IL

1️⃣ Why people say this sentence at all

When someone says:

> “struct is considered like a class before”

They usually mean **one of these two things** (not both):

Meaning A (most common)

> *“A struct can look syntactically like a class.”*

Example:

```
struct Point { public int X; public int Y; public void Move() { } }
```

It **looks like a class**:

- Fields
- Methods
- Access modifiers
- Interfaces

📌 This is **syntax-level similarity**, not runtime behavior.

Meaning B (sloppy explanation)

> *“Struct behaves like a class at first glance.”*

Especially when passed to methods or stored in variables.

This is **misleading** unless explained carefully.

2️⃣ The fundamental truth (THIS is the core)

❗ In CLR:

> **class = reference type**
> **struct = value type**

This difference is **absolute**, deep, and non-negotiable.

3️⃣ What is ACTUALLY similar between struct and class

✓ Same **syntax features**:

- Fields
- Methods
- Properties
- Constructors
- Interfaces

✓ Same **IL metadata presence**

✓ Same **type checking rules**

That’s it.

4️⃣ What is COMPLETELY DIFFERENT (critical)

Aspect	struct	class
CLR category	ValueType	ReferenceType

Aspect	struct	class
Allocation	Inline	Heap
Copy behavior	Copy by value	Copy reference
Nullability	Cannot be null	Can be null
Inheritance	No (only interfaces)	Yes
Identity	No identity	Has identity
Lifetime	Scope-based	GC-based